LENGUAJE DE PROGRAMACIÓN II · SESIÓN 13 · ACTIVIDAD AUTÓNOMA

Informe de optimización y preparación para producción de PharmaSoft

Reto 02 · Evaluación de carga diferida, caché HTTP, Redis, trazabilidad, seguridad, monitoreo y despliegue

Ficha del trabajo	Detalle
Proyecto	PharmaSoft (Angular + Spring Boot + Oracle + Redis + Nginx)
Asignatura	Lenguaje de Programación II - ciclo IV, semestre 2026-2
Documento base	Actividad Autónoma - Sesión 13 (Reto 02), 5 de noviembre de 2026
Entrega prevista	11 de noviembre de 2026, 23:59
Tipo de evaluación	Pruebas técnicas reproducibles en entorno Docker local
Estado del informe	Resultados comprobados y puntos de entrega aún por acreditar

Nota de integridad. Este documento recoge únicamente cifras, comandos y resultados compartidos durante la ejecución de las pruebas. Las capturas se presentan recortadas para evitar credenciales y datos personales. Los requisitos que aún no cuentan con evidencia se declaran expresamente.

Nombre de entrega: Zambrano_LP2_S13_Autonoma.pdf

Resumen ejecutivo

Se evaluó PharmaSoft en un entorno local compuesto por SPA Angular 22, Nginx, backend Spring Boot con Java 21, Oracle y Redis. Se aplicó carga diferida al diseño principal, se verificaron encabezados HTTP y actualización de archivos con hash, se ejecutaron pruebas de Redis y se incorporó un identificador de seguimiento a los mensajes de error del frontend.

Dimensión	Resultado verificado	Interpretación
Tamaño inicial	310.30 → 301.64 kB (-8.66 kB; -2.79 %)	Menos JavaScript inicial mediante carga diferida de MainLayout.
Precarga	146 → 175 kB transferidos en primera carga	PreloadAllModules incrementó la transferencia inicial bajo las condiciones de la prueba.
Redis	39 hits / 3 misses = 92.86 %	Tasa global de métricas acumuladas; 42 accesos registrados.
TTL	60 → -2/0 → 589 segundos	Expiración de una clave y nueva creación con TTL próximo a 10 min.
Venta	Stock 16 → 15 unidades	El stock se actualizó tras una venta de una unidad.
Trazabilidad	Mismo requestId en cabecera, JSON y log	Permite correlacionar errores y operaciones internas.
Disponibilidad	Health: 503 sin Redis / 200 recuperado	Se comprobó detección de caída y recuperación.

Alcance y método

Las pruebas se ejecutaron con compilaciones de producción, consola PowerShell, Docker Compose, redis-cli, Chrome DevTools y Spring Boot Actuator. Las mediciones en navegador incluyen latencia de red local y carga de la aplicación; no deben interpretarse como benchmarks aislados del servidor.

Limitaciones principales: no quedó registrado el tiempo visual exacto hasta mostrar Ventas; la prueba de TTL a un minuto se realizó sobre una clave con EXPIRE (no mediante cambio de la propiedad global); la descarga de cada archivo en su primera visita posterior al despliegue no se capturó de forma independiente; los requisitos de versionamiento en tres días distintos aún no están acreditados.

Parte A. Carga diferida y tamaño de entrega

A.1 Análisis del paquete

Se generó la compilación de producción con información estadística y se inspeccionó la representación tipo treemap del analizador esbuild. La siguiente tabla recoge los cinco módulos de mayor tamaño visibles dentro de los bloques iniciales del analizador. Las magnitudes son aproximadas, leídas de la visualización, y no equivalen a tamaños comprimidos descargados.

Módulo visible en analizador	Origen / dependencia	Tamaño aprox.
_debug_node-chunk.mjs	@angular/core / runtime Angular	132.4 kB
_router-chunk.mjs	@angular/router / navegación y rutas	85.3 kB
_pending_tasks-chunk.mjs	@angular/core / planificación de tareas	34.3 kB
_module-chunk.mjs	@angular/common / módulo común	24.5 kB
_common_module-chunk.mjs	@angular/common / componentes y directivas	21.2 kB

El analizador también muestra RxJS (dist, alrededor de 20.3 kB), código propio de Ventas y Productos y módulos de formularios. Estos últimos aparecen principalmente en fragmentos diferidos, por lo que no todos pertenecen al paquete inicial.

A.2 Mejora adicional aplicada y medida

Indicador	Antes	Después	Variación
Initial total (raw)	310.30 kB	301.64 kB	-8.66 kB (-2.79 %)
Transferencia estimada	87.50 kB	85.04 kB	-2.46 kB (-2.81 %)
Fragmentos lazy	15	16	+1 fragmento
MainLayout	Carga inmediata	loadComponent()	~8.88 kB diferidos

La mejora consistió en sustituir la importación estática de **MainLayout** por **loadComponent(() => import(...))** en app.routes.ts. La comparación procede de dos compilaciones de producción. El tamaño inicial se redujo sin alterar las rutas funcionales de Inicio, Categorías, Productos y Ventas.

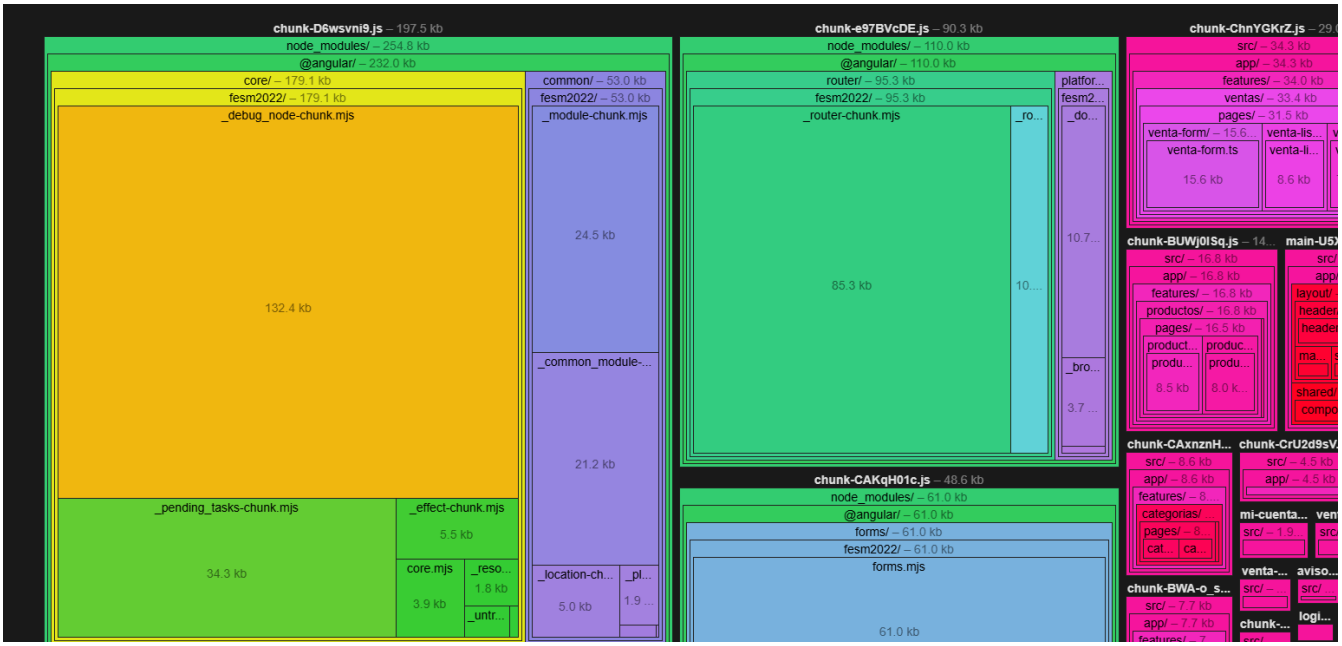
A.3 Ensayo con y sin precarga

Medición con DevTools	Sin precarga	PreloadAllModules
Primera carga (solicitudes)	21	31
Primera carga (transferidos)	146 kB	175 kB
Primera carga (recursos)	377 kB	444 kB
Entrada a Ventas (solicitudes)	7	4
Entrada a Ventas (transferidos)	15.8 kB	3.0 kB
Entrada a Ventas (recursos)	37.0 kB	5.7 kB

Decisión: conservar carga diferida sin precarga global. En la muestra, la precarga añadió 29 kB a la primera carga y redujo 12.8 kB al abrir Ventas. Se prioriza una entrada inicial menos pesada para locales con conectividad variable. No se registró de forma comparable el tiempo exacto hasta que Ventas quedó visible; esa dimensión no se declara demostrada.

Evidencia A. Analizador de paquetes

Figura A-01. Treemap de dependencias esbuild



Captura original del analizador: identifica paquetes Angular (core, common, router y forms), RxJS y fragmentos de código propio. Las cifras de la tabla A.1 se leen de este gráfico.

Los pesos observados corresponden al estado de análisis de paquetes; la comparación cuantitativa de tamaño inicial se fundamenta en las salidas de compilación con y sin la mejora.

Parte B. Política de caché del navegador

El frontend se sirvió en el host local **http://localhost:8082** mediante Nginx. Se consultaron encabezados con curl y se comprobó el comportamiento con DevTools. El puerto difiere del ejemplo de la guía porque Docker Compose publica el frontend en 8082.

Recurso	Cache-Control observado	Recarga/resultado	Evaluación
/index.html	no-cache	Revalidación HTTP 304	Correcto
main-*.js	public, max-age=31536000, immutable	Memory cache (200)	Correcto
chunk-* Ventas	public, max-age=31536000, immutable	Memory cache (200)	Correcto
styles-*.css	public, max-age=31536000, immutable	Caché persistente aplicable; no se capturó fuente exacta	Correcto por cabecera
/api/v1/categorias	no-cache, no-store, max-age=0, must-revalidate	Solicitudes fetch 200; prueba inicial sin token 401	Correcto

En una recarga de Ventas con la caché del navegador habilitada se observaron **16 solicitudes, 2.5 kB transferidos y 397 kB de recursos**. Los ficheros JS procedieron de memoria; el documento de ruta respondió con 304. Las respuestas API no se almacenan porque contienen datos dinámicos, incluidos precios, categorías y existencias, sujetos a cambios inmediatos.

B.1 Simulación de un despliegue sin detener Nginx

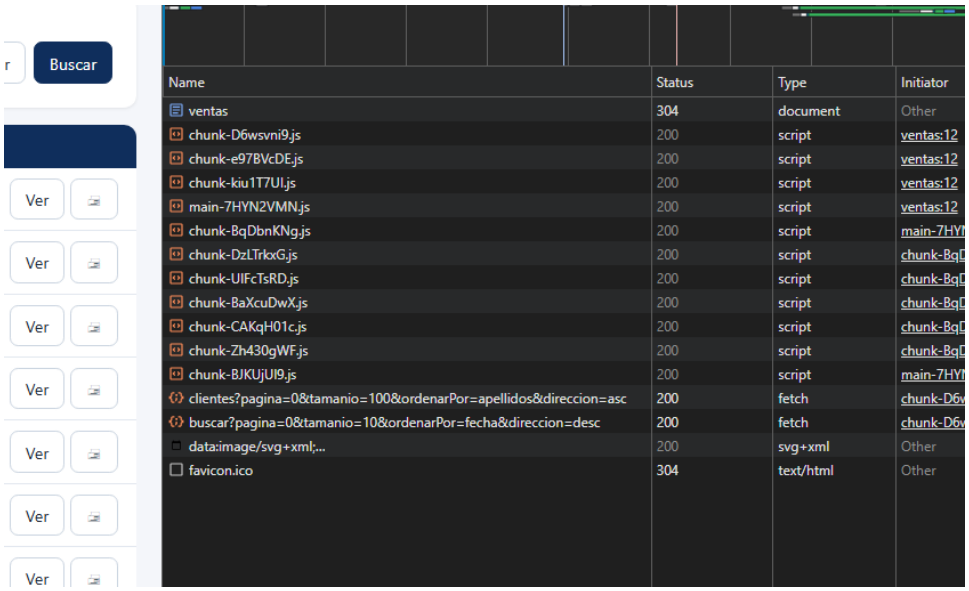
Se cambió temporalmente el título visible de Productos por «Productos - Version S13». Tras compilar, los archivos se copiaron al contenedor Nginx en ejecución, colocando el index.html al final para que apuntase a los nuevos hashes. El servidor respondió HTTP 200 y mantuvo index.html con no-cache.

Archivo	Versión anterior	Nueva versión
main	main-7HYN2VMN.js	main-QYCZV62H.js
Productos (lazy)	chunk-Xv7i2B-7.js	chunk-7bQfB4oS.js

La nueva pantalla se visualizó sin limpiar la caché. El navegador revalida el HTML y resuelve los nuevos archivos con hash. La captura final mostró recursos servidos desde memory cache; no permite reconstruir cuáles se transfirieron por red la primera vez. Finalizada la prueba, se restauró el texto «Productos» y se reconstruyó el contenedor.

Evidencias B. Caché y nuevo despliegue

Figura B-01. Archivos JS recuperados desde memoria



Chrome DevTools: la mayoría de los archivos JavaScript se atiende desde memory cache, con transferencia total reducida y respuesta 304 del documento.

Figura B-02. Versión S13 visible en Productos



Cambio temporal de título mostrado tras desplegar los nuevos artefactos en Nginx. Posteriormente se restauró el título original.

Parte C. Redis: aciertos, vigencia e invalidación

C.1 Aciertos y fallos

Se reinició el backend, se eliminaron las claves de prueba y se lanzaron 20 solicitudes autenticadas de categorías y 20 de productos desde Chrome. La medición se obtuvo de /actuator/metrics/cache.gets, filtrando por los nombres de caché y por result.

Caché	Hits	Misses	Pending	Tasa de aciertos
categorias	20	1	0	95.24 %
productos	19	2	0	90.48 %
Total	**39**	**3**	**0**	**92.86 %**

Cálculo: tasa = hits / (hits + misses) × 100. Los contadores suman **42 accesos**, no exactamente 40, debido a solicitudes adicionales contabilizadas por Actuator. Por ello se informa la tasa acumulada de la caché, no una tasa exclusiva de las 40 peticiones del bucle.

C.2 Tiempos observados

Para Productos, la primera solicitud registró **76.3 ms**. Las 19 siguientes registradas en la tabla del navegador promediaron aproximadamente **19.94 ms**, es decir, la primera tardó 3.83 veces el promedio posterior. En Categorías se observaron respuestas posteriores próximas a 17–20 ms, pero no se dispone de una medición individual completa del primer fallo para un promedio comparable.

C.3 Vigencia de la caché

Comprobación	Respuesta Redis	Conclusión
TTL previo productos	292 s	Quedaban ~4 min 52 s.
EXPIRE productos 60	1; TTL 60	Se fijó expiración de 60 s en una clave.
Después de expirar	TTL -2; EXISTS 0	La clave ya no existía.
Nueva consulta de Productos	EXISTS 1; TTL 589	Redis recreó la clave con TTL cercano a 600 s.

Alcance: la configuración general time-to-live: 10m permaneció intacta. El ensayo de un minuto se realizó mediante el comando redis-cli EXPIRE sobre una entrada concreta, como alternativa de demostración; no representa un cambio del TTL global de la aplicación.

C.4 Matriz de invalidación y consistencia

Caso	Operación	Dato comprobado	Resultado
C-01	Crear categoría temporal	PRUEBA CACHE S13, ID 81, visible	Correcto
C-02	Renombrar categoría	PRUEBA CACHE S13 EDITADA visible en filtro Productos	Correcto
C-03	Modificar precio	Multivitamínico Adulto: S/ 30 → S/ 31	Correcto; precio restaurado
C-04	Registrar venta de 1 unidad	Stock Multivitamínico Adulto: 16 → 15	Correcto
C-05	Eliminar categoría sin productos	ID 81 desapareció; operación HTTP 204; TTL -2	Correcto
C-06	Detener Redis	Productos responde 200; health 503; vuelve a 200 tras recuperación	Correcto

Consistencia: la creación, edición y eliminación se reflejaron en las vistas. El stock se descontó conforme a la venta registrada. La eliminación de la categoría C-05 devolvió TTL -2, compatible con invalidación. En C-01 a C-04, los TTL próximos a 600 segundos son compatibles con reconstrucción de caché, pero por sí solos no identifican el instante de evicción.

Defectos: no se reprodujo una inconsistencia de datos que justificase corregir @CacheEvict. La prueba C-06 mostró degradación controlada: la lectura de Productos siguió disponible sin Redis y Actuator informó la dependencia caída.

Gestión de registros de prueba: la categoría temporal ID 81 se eliminó; el precio del producto se restauró a S/ 30.00. La venta quedó registrada y dejó una reducción legítima de stock de una unidad.

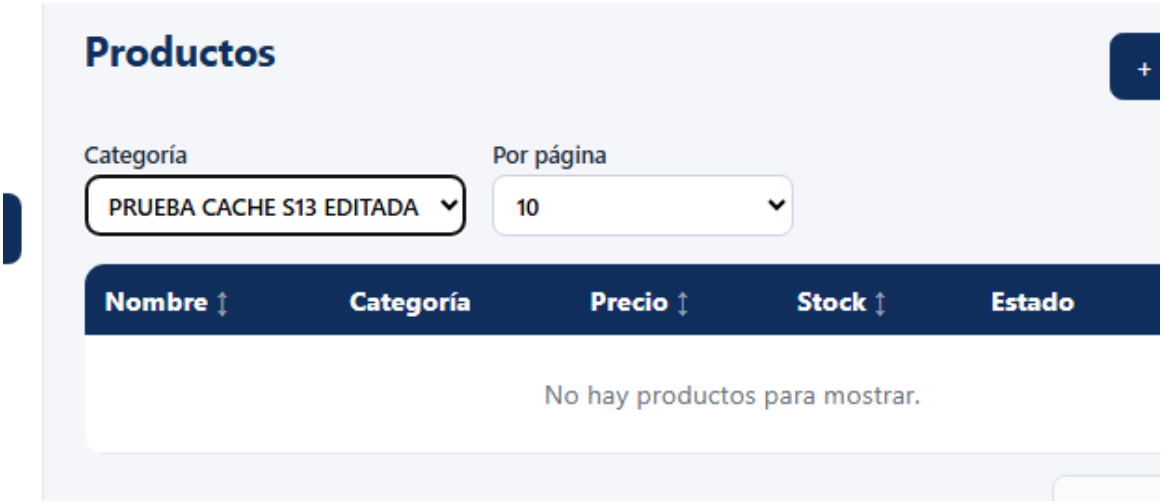
Evidencias C. Categorías y precio (C-01 a C-03)

Figura C-01. C-01: categoría temporal registrada



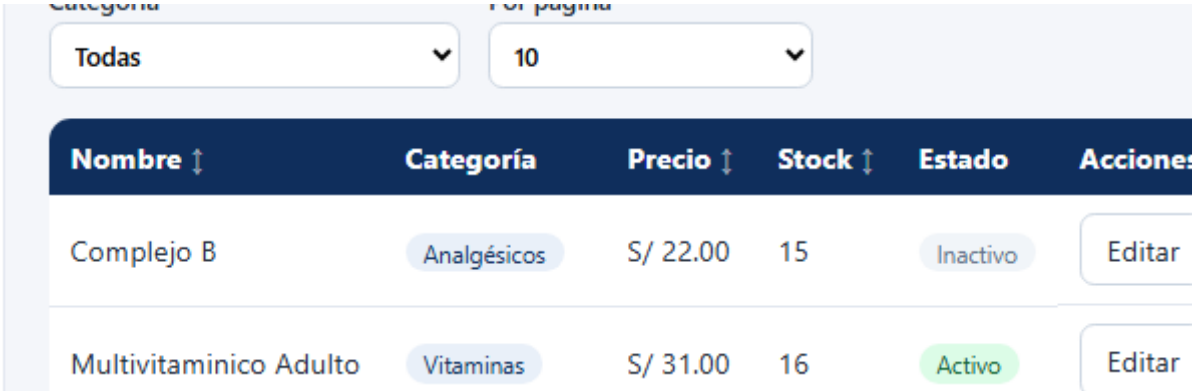
La categoría PRUEBA CACHE S13 aparece en el listado después de crearla.

Figura C-02. C-02: categoría editada en Productos



El filtro de Productos muestra PRUEBA CACHE S13 EDITADA, sin productos asociados.

Figura C-03. C-03: precio modificado



Multivitamínico Adulto muestra S/ 31.00. Se restauró posteriormente a S/ 30.00.

Evidencias C. Venta, borrado y fallo de Redis

Figura C-04. C-04: stock actualizado



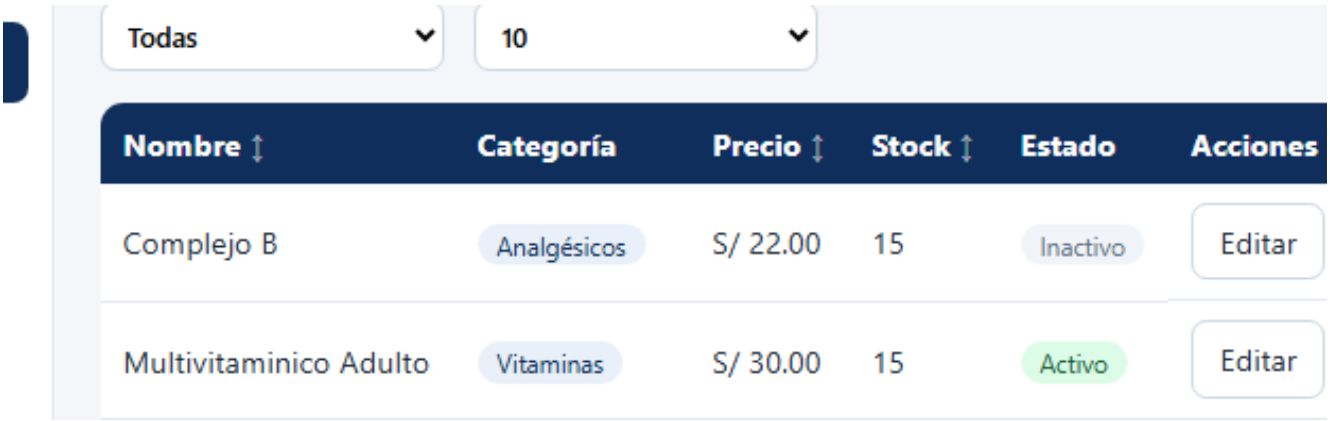
Producto Multivitamínico Adulto con 15 unidades tras una venta de una unidad; antes tenía 16.

Figura C-05. C-05: categoría eliminada



Listado sin la categoría temporal ID 81; la respuesta de eliminación fue HTTP 204 y Redis registró TTL -2.

Figura C-06. C-06: Productos disponible sin Redis



Se capturó el listado de productos durante la desconexión de Redis; Actuator respondió 503 en esa condición.

Parte D. Logs, seguimiento y monitoreo

D.1 Correlación de la consulta de venta

Para la venta N.º 91, la pestaña Network mostró el encabezado **X-Request-Id: 6c14a5dd**. La búsqueda en docker logs confirmó tres eventos con ese mismo identificador:

Evento	Extracto del log	Interpretación
Inicio	[6c14a5dd] Inicio buscar venta por id | id=91	Entrada a la operación.
Fin	[6c14a5dd] Fin buscar venta por id | filas=1 | duracionMs=6	Resultado interno y tiempo de servicio.
HTTP	[6c14a5dd] GET /api/v1/ventas/91 -> 200 (25 ms)	Resultado final y tiempo de solicitud.

D.2 Identificador en respuestas de error y en la SPA

Se incorporó requestId al DTO de error tomando MDC.get("requestId") en su constructor de compatibilidad. La prueba contra /api/v1/ventas/99999999 produjo **HTTP 404** y el mismo ID **f47f9794** en el JSON y X-Request-Id (coinciden = true). Posteriormente Angular mostró el error con **«Código de seguimiento: 898af874»** para una solicitud independiente. Esto permite que soporte localice el evento exacto sin solicitar credenciales ni datos personales.

D.3 Revisión de secretos en logs

Búsqueda	Coincidencias	Interpretación
Bearer	0	No se detectaron cabeceras Bearer.
JWT con tres segmentos	0	Sin JWT completos detectados.
password / secret = valor	0	Sin contraseñas o secretos en el patrón revisado.
Cadenas de 8 dígitos	5	1 código HHH de Hibernate y 4 referencias a la venta ficticia 99999999.

La revisión de las coincidencias de ocho dígitos confirmó que no eran DNI. Esta auditoría cubre los logs disponibles en la ventana de dos horas consultada; no es una certificación de que ninguna ruta futura pueda registrar información sensible.

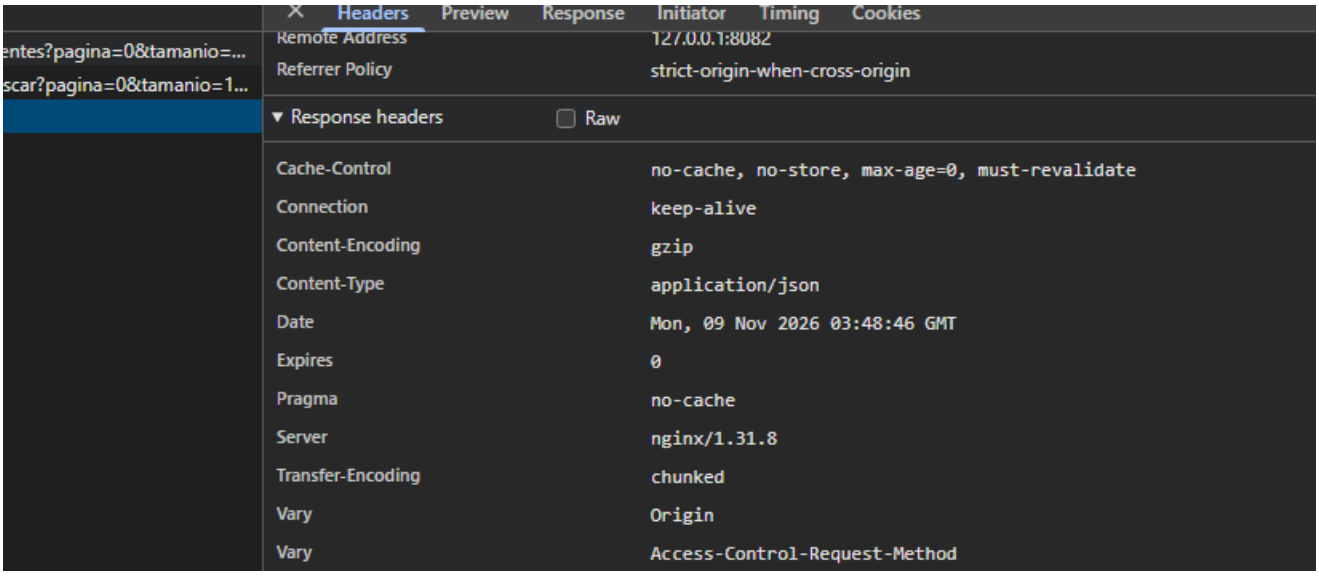
D.4 Health y métricas

Verificación	Resultado	Lectura técnica
Health sin token, Redis activo	HTTP 200 / UP	Disponibilidad pública observada.
Health con Redis detenido	HTTP 503	Dependencia fallida reportada.
Health tras recuperación	HTTP 200 / UP	Servicio recuperado.
Métricas sin token	HTTP 401	Acceso controlado por autenticación.
Métricas con token ADMIN	HTTP 200	Métricas accesibles autenticadas.
http.server.requests uri=/api/v1/ventas/{id}	COUNT 2; TOTAL_TIME 0.295378764 s; MAX 0.0	Contador específico de solicitudes de detalle de venta.

El valor MAX = 0.0 fue devuelto tal cual por Actuator; no se interpreta como una latencia máxima real. Se recomienda supervisar contadores y percentiles en una ventana temporal apropiada antes del despliegue público.

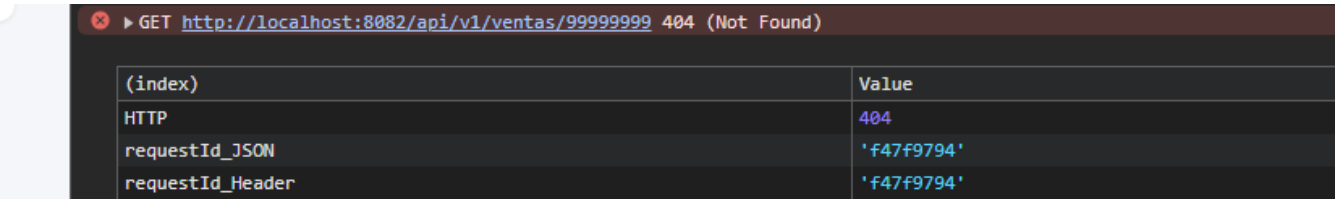
Evidencias D. Correlación y errores

Figura D-01. D-01: X-Request-Id de venta 91



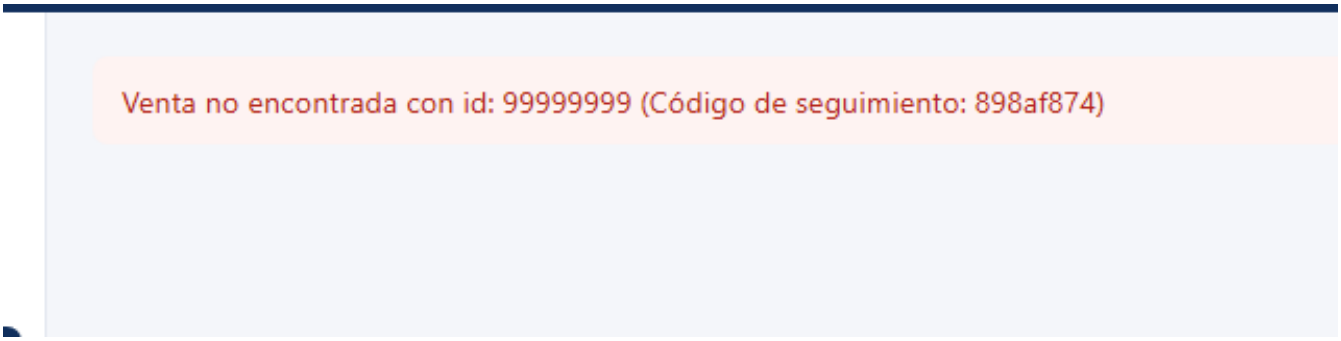
La respuesta HTTP incluye X-Request-Id 6c14a5dd; coincide con las tres líneas de log transcritas en D.1.

Figura D-02. D-02: igualdad entre JSON y cabecera



En la consola del navegador se verifica que el requestId del cuerpo JSON y el de Response Headers son idénticos.

Figura D-03. D-03: código de seguimiento visible en la SPA



Angular muestra el error de venta inexistente junto a «Código de seguimiento: 898af874».

Parte E. Preparación para producción

E.1 Arquitectura propuesta

Los README de backend y frontend se crearon/actualizaron con instrucciones de compilación, variables, operación Docker, recuperación y prioridades. La topología de despliegue propuesta es:

Origen	Componente	Destino / función
Usuario / navegador	Nginx + Angular (host local 8082)	Archivos SPA con hash; reverse proxy /api/
Nginx	Spring Boot (host local 8081)	API REST, JWT, logs, Actuator
Spring Boot	Oracle (red Docker; DB_URL)	Datos transaccionales, categorías, productos y ventas
Spring Boot	Redis (6379 local)	Caché de lectura, invalidación y TTL

Para instalar PharmaSoft en tres locales, los puertos internos deben aislarse y publicarse exclusivamente a través de un acceso HTTPS controlado. La red, las copias de seguridad, los límites de acceso administrativo y la configuración de secretos deben ser definidos para el entorno real.

E.2 Variables sin valores reales

Variable	Propósito
DB_URL	Cadena de conexión al servicio Oracle.
DB_USERNAME	Cuenta de acceso a la base de datos.
DB_PASSWORD	Secreto de autenticación a Oracle.
JWT_SECRET	Clave de firma de tokens de acceso.
FRONTEND_ORIGINS	Orígenes permitidos en la política CORS.

E.3 Construcción, despliegue y reversión

• Construir los artefactos con npm run build -- --configuration production y con la imagen Maven/Java 21 del backend.

• Validar Docker Compose, health, permisos, acceso a Oracle y consultas con Redis habilitado antes de cambiar el tráfico.

• Conservar la etiqueta de la imagen previa y una copia respaldada de Oracle; si ocurre un fallo, volver a la imagen estable y revalidar transacciones.

• No asumir que revertir el código revierte migraciones o datos: planificar compatibilidad del esquema y respaldo/restauración.

E.4 Lista de cotejo para tres locales

Práctica	Estado	Acción pendiente para producción
Compilación optimizada Angular	Cumple	Mantener registro del tamaño y pruebas automáticas.
Archivos con hash y política HTTP	Cumple	Revalidar políticas con HTTPS y CDN, si aplica.
Redis con invalidación y recuperación	Cumple parcialmente	Centralizar configuración, proteger Redis y agregar supervisión.
Logs con requestId	Cumple	Revisar retención, permisos y sanitización periódica.
Auditoría de datos sensibles	Cumple parcialmente	Extender pruebas a logs históricos y nuevos endpoints.
Monitoreo y alertas	Cumple parcialmente	Añadir alertas activas y panel de métricas.
HTTPS público	No cumple / no acreditado	Configurar TLS antes de exposición externa.
Backups probados de Oracle	No acreditado	Automatizar y ensayar restauraciones.
Rollback verificable	Cumple parcialmente	Ensayar recuperación completa con tags y migraciones.
Versionado y PR a develop	Pendiente de cierre	Verificar commits por fecha, push y solicitudes de incorporación.

E.5 Recomendaciones priorizadas

• **Prioridad 1 - TLS, gestión de secretos y control de acceso.** Usar HTTPS extremo a extremo, restringir Actuator, no publicar secretos ni endpoints internos de Oracle y Redis.

• **Prioridad 2 - Respaldo y recuperación.** Programar copias de seguridad de Oracle y comprobar periódicamente su restauración; definir RPO y RTO de los tres locales.

• **Prioridad 3 - Observabilidad y disponibilidad.** Centralizar logs asociados a requestId y generar alertas de health, errores HTTP, latencias y disponibilidad de Redis y Oracle.

Estos controles son recomendaciones previas al despliegue en producción. El funcionamiento satisfactorio en Docker local no implica que estén implementados en la infraestructura de los tres locales.

Parte F. Repositorios, versión y entrega

La revisión de los repositorios mostró la rama **feature/optimizacion-produccion-zambrano** tanto en frontend como en backend. Los archivos para el cambio estaban preparados en staging, pero no consta en la conversación una confirmación posterior de los commits ni de los pushes.

Repositorio	Cambios preparados	Evidencia de fechas
Backend	5 archivos: README, RequestIdFilter, ErrorResponseDTO, application-dev.yaml, application-prod.yaml	Commits visibles: 08/11 y 19/10; falta acreditar 3 fechas distintas.
Frontend	3 archivos: README, app.routes.ts, core/utils/http-error.ts	Los últimos tres commits mostrados son del 08/11; no se acredita el requisito de tres días.

F.1 Entregables de este informe

• **Informe principal:** Zambrano_LP2_S13_Autonoma.pdf para aula virtual.

• **Copia técnica:** informe-s13.pdf destinada a docs/informe-s13.pdf en el backend.

• **Capturas:** evidencias A, B, C y D insertadas en este documento y recortadas para no exhibir datos personales.

F.2 Verificaciones pendientes antes de la entrega

• Confirmar al menos tres commits efectuados en días reales distintos en cada repositorio; no alterar fechas retrospectivamente.

• Confirmar push y solicitudes de incorporación hacia develop en ambos repositorios y resolver posibles conflictos.

• Copiar informe-s13.pdf a docs/ dentro del backend, ejecutar git add/commit/push y subir Zambrano_LP2_S13_Autonoma.pdf al aula virtual.

• Revisar de nuevo que las capturas y archivos publicados no contengan tokens, contraseñas o DNI; realizar revisión de secretos de todos los cambios antes de push.

• Si el docente exige el cambio global a TTL 1 minuto, repetir ese ensayo modificando temporalmente time-to-live y restaurando 10m después.

Estado: el documento técnico y sus pruebas observadas se encuentran elaborados. El cierre formal de la Parte F depende de actuaciones en Git y en el aula virtual que no se han podido verificar desde este entorno.

Anexo. Protocolo reproducible y control de datos

Construcción Angular y análisis

```
npm run build -- --configuration production --stats-json
```

```
Abrir dist/pharma-frontend/browser-stats.json en esbuild.github.io/analyze
```

```
Comparar Initial total y las solicitudes en DevTools (Disable cache según el ensayo).
```

Cabeceras HTTP y caché Redis

```
curl.exe -I http://localhost:8082/index.html
```

```
curl.exe -I http://localhost:8082/main-7HYN2VMN.js
```

```
docker exec pharma-redis redis-cli TTL "productos::0-10-nombre-asc"
```

```
docker exec pharma-redis redis-cli EXPIRE "productos::0-10-nombre-asc" 60
```

```
docker exec pharma-redis redis-cli EXISTS "productos::0-10-nombre-asc"
```

Actuator y logs

```
docker logs pharma-backend --since 30m | Select-String "6c14a5dd"
```

```
GET /actuator/metrics/cache.gets?tag=cache:productos&tag=result:hit
```

```
GET /actuator/metrics/http.server.requests?tag=uri:/api/v1/ventas/{id}
```

```
curl.exe -s -i http://localhost:8081/actuator/health
```

Privacidad: las capturas integradas se recortaron para evitar nombres de usuarios, DNI y cabeceras de autorización. Las salidas de comandos con token nunca se insertaron en el informe. Las pruebas de creación y venta sí modificaron la base de datos local y deben registrarse como acciones realizadas, no como simples simulaciones visuales.

Fuente de información: guía oficial de Actividad Autónoma, Sesión 13, Reto 02, y evidencias técnicas compartidas durante las pruebas del proyecto PharmaSoft el 8 y 9 de noviembre de 2026. No se añadieron resultados no observados.